Taller Docker – Semana 13

Contenerización segura y auditoría forense de **eco-air-backend**

Nicolás Bastidas
STUDENT_ID / OPERATOR_ID: 890275

Ingeniería de Software
21 de octubre de 2026

1. Objetivo

Empaquetar el backend **eco-air-backend** (Spring Boot 3.3.5, Java 17, JPA + H2) en una imagen Docker liviana y segura, comprobar con pruebas forenses que la imagen final no tiene herramientas de compilación ni código fuente y que no corre como root, y encontrar el límite mínimo de memoria con el que el servicio funciona de forma estable.

2. Entorno de la auditoría

```
○ ○ ○  Terminal — aleyport@fedora:~/eco-air-backend

[2026-10-21 12:43:05] $ date
mié 21 oct 2026 12:43:05 -05
(exit code: 0)

[2026-10-21 12:43:05] $ docker --version
Docker version 29.6.2, build 1.fc43
(exit code: 0)

[2026-10-21 12:43:05] $ docker info --format 'Server: {{.ServerVersion}} | OS:
{{.OperatingSystem}} | Kernel: {{.KernelVersion}} | CPUs: {{.NCPU}} | RAM: {{.MemTotal}} |
Cgroup: {{.CgroupVersion}}'
Server: 29.6.2 | OS: Fedora Linux 43 (KDE Plasma Desktop Edition) | Kernel:
7.1.12-100.fc43.x86_64 | CPUs: 12 | RAM: 33310220288 | Cgroup: 2
(exit code: 0)
```

La prueba se hizo en Fedora Linux 43 con Docker Engine 29.6.2 nativo y **cgroup v2**. Al no haber una capa de virtualización (como WSL2), los límites de memoria se aplican directamente con el kernel del equipo.

3. Reto 1 – Construcción multietapa

3.1 Dockerfile

Dockerfile

```
# ---------- Etapa 1: builder ----------
FROM maven:3.9.6-eclipse-temurin-17-alpine AS builder
ARG STUDENT_ID
WORKDIR /build

# Solo el pom primero -> la capa de dependencias queda en caché
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Ahora el código fuente
COPY src ./src
RUN echo "Build por: ${STUDENT_ID}" && mvn package -DskipTests -B

# ---------- Etapa 2: runtime ----------
FROM eclipse-temurin:17-jre-alpine
# ARG se pierde entre etapas: hay que redeclararlo
ARG STUDENT_ID
LABEL operator.id="${STUDENT_ID}"

RUN addgroup -S sensorgroup && \
    adduser -S -D -H -u 10001 -G sensorgroup sensor_user

WORKDIR /app
COPY --from=builder /build/target/*.jar app.jar
RUN chown -R sensor_user:sensorgroup /app

ENV OPERATOR_ID=${STUDENT_ID} \
    JAVA_TOOL_OPTIONS="-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0
-XX:InitialRAMPercentage=50.0 -XX:+UseSerialGC -XX:+ExitOnOutOfMemoryError"

USER sensor_user
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

3.2 Justificación de las decisiones

Dos etapas (builder → runtime). La primera etapa usa la imagen de Maven con JDK para descargar dependencias y compilar. La segunda parte de una imagen que solo tiene el JRE y recibe únicamente el `app.jar`. Maven, el JDK, el `pom.xml` y el código fuente se quedan en la etapa builder y no llegan a producción. Así la imagen es más pequeña y un atacante tiene menos herramientas a la mano.

Caché de dependencias. Primero se copia solo el `pom.xml` y se ejecuta `mvn dependency:go-offline`. Si después solo cambia el código, Docker reutiliza esa capa y no vuelve a descargar las dependencias.

ARG redeclarado. Un `ARG` solo existe dentro de la etapa donde se declara, por eso `STUDENT_ID` se vuelve a declarar en la etapa runtime. Ahí se usa para el `LABEL operator.id` y para la variable `OPERATOR_ID`.

Usuario no root. Se crea `sensor_user` con UID fijo 10001, sin contraseña ni home, y se activa con `USER` antes del `ENTRYPOINT`. Si alguien compromete la aplicación, queda con permisos de usuario normal y no puede modificar el sistema.

Opciones de la JVM (variable `JAVA_TOOL_OPTIONS`):

Opción	Para qué sirve
-XX:+UseContainerSupport	La JVM lee el límite de memoria del cgroup en lugar de la RAM total del equipo.
-XX:MaxRAMPercentage=75.0	El heap puede usar hasta el 75 % del límite. El resto queda para metaspace, hilos y código nativo.
-XX:InitialRAMPercentage=50.0	El heap empieza en el 50 % del límite, lo que evita crecer muchas veces durante el arranque.
-XX:+UseSerialGC	Recolector de basura de un solo hilo. Gasta menos memoria propia que G1 y conviene en contenedores pequeños.
-XX:+ExitOnOutOfMemoryError	Si se acaba el heap, la JVM termina en vez de quedar en un estado inestable, y Docker puede reiniciarla.

3.3 .dockerignore

○○○ .dockerignore

```
target/
.git/
.gitignore
.idea/
.vscode/
*.iml
*.log
*.md
Dockerfile
.dockerignore
.mvn/wrapper/maven-wrapper.jar
evidencias/
run_taller.*
```

El `.dockerignore` evita enviar al contexto de build carpetas como `target/` o `.git/`, además de logs, scripts y evidencias. La build es más rápida y no se filtran archivos que no hacen falta.

3.4 Resultado de la build

○○○ Terminal — aleyport@fedora:~/eco-air-backend

```
$ docker build --build-arg STUDENT_ID="890275" -t eco-air-backend:1.0.0 .
...
#14 19.24 [INFO] Building jar: /build/target/eco-air-backend.jar
#14 DONE 20.0s
#17 naming to docker.io/library/eco-air-backend:1.0.0 done
#17 DONE 7.3s
(exit code: 0)

$ docker images eco-air-backend:1.0.0
IMAGE                    ID             DISK USAGE   CONTENT SIZE
eco-air-backend:1.0.0    568e9b8ed17b        453MB          159MB

$ docker image inspect eco-air-backend:1.0.0 --format 'Labels: {{json .Config.Labels}} | User:
{{.Config.User}}'
Labels: {"operator.id":"890275"} | User: sensor_user
```

La imagen se construyó sin errores. Pesa 159 MB de contenido y quedó con el label `operator.id=890275` y el usuario `sensor_user`.

4. Reto 2 – Auditoría forense

4.1 Historial de capas

```
Terminal — aleyport@fedora:~/eco-air-backend

[2026-10-21 12:53:36] $ docker history --human --format 'table {{.Size}}\t{{.CreatedBy}}' eco-
air-backend:1.0.0
SIZE      CREATED BY
0B        ENTRYPOINT ["java" "-jar" "/app/app.jar"]
0B        EXPOSE [8080/tcp]
0B        USER sensor_user
0B        ENV OPERATOR_ID=890275 JAVA_TOOL_OPTIONS=-XX…
48.9MB    RUN |1 STUDENT_ID=890275 /bin/sh -c chown -R…
48.9MB    COPY /build/target/*.jar app.jar # buildkit
0B        WORKDIR /app
24.6kB    RUN |1 STUDENT_ID=890275 /bin/sh -c addgroup…
0B        LABEL operator.id=890275
0B        ARG STUDENT_ID=890275
0B        ENTRYPOINT ["/__cacert_entrypoint.sh"]
8.19kB    COPY --chmod=755 entrypoint.sh /__cacert_ent…
0B        RUN /bin/sh -c set -eux;     echo "Verifying…
142MB     RUN /bin/sh -c set -eux;     ARCH="$(apk --p…
0B        ENV JAVA_VERSION=jdk-17.0.20.1+1
44.8MB    RUN /bin/sh -c set -eux;     apk add --no-ca…
0B        ENV LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_AL…
0B        ENV PATH=/opt/java/openjdk/bin:/usr/local/sb…
0B        ENV JAVA_HOME=/opt/java/openjdk
0B        CMD ["/bin/sh"]
10.1MB    ADD alpine-minirootfs-3.24.2-x86_64.tar.gz /…
(exit code: 0)
```

Qué demuestra: todas las capas vienen de Alpine, del JRE 17 o de los pasos de la etapa runtime. No hay ninguna capa con `mvn`, `pom.xml` ni `src/`, así que la etapa builder quedó descartada.

4.2 Proceso visto desde el host

```
Terminal — aleyport@fedora:~/eco-air-backend

[2026-10-21 12:53:36] $ docker run -d --name eco_air_active -p 8080:8080 eco-air-backend:1.0.0
48f0844f3350d248d53da35c793865c5f27bda47361dcd4e4dc36e72bcd9cdd0
(exit code: 0)

[2026-10-21 12:53:44] $ docker top eco_air_active -o pid,user,comm
PID                 USER                COMMAND
120340              10001               java
(exit code: 0)

[2026-10-21 12:53:44] $ docker exec eco_air_active id
uid=10001(sensor_user) gid=101(sensorgroup) groups=101(sensorgroup)
(exit code: 0)

[2026-10-21 12:53:44] $ curl -s http://localhost:8080/api/health
{"status":"UP","operator":"890275"}
(exit code: 0)
```

Qué demuestra: desde el host, el proceso `java` corre con el UID **10001** y no con 0 (root). Dentro del contenedor, `id` confirma que es `sensor_user`. El endpoint de salud responde con el OPERATOR_ID.

4.3 Prueba de inmutabilidad (penetración)

```
○ ○ ○  Terminal — aleyport@fedora:~/eco-air-backend

[2026-10-21 12:53:44] $ docker exec eco_air_active touch /etc/crontab_fake
touch: cannot touch '/etc/crontab_fake': Permission denied
(exit code: 1)

[2026-10-21 12:53:44] $ docker exec eco_air_active ls -ld /etc /app
drwxr-xr-x 1 sensor_user sensorgroup 14 Oct 21 17:53 /app
drwxr-xr-x 1 root        root        56 Oct 21 17:53 /etc
(exit code: 0)
```

Qué demuestra: crear un archivo falso en `/etc` (como lo haría un atacante para dejar un cron) falla con **Permission denied**. `/etc` pertenece a root, y la app solo puede escribir en `/app`.

4.4 Artefactos de desarrollo

```
○ ○ ○  Terminal — aleyport@fedora:~/eco-air-backend

[2026-10-21 12:53:44] $ docker exec eco_air_active which mvn
(exit code: 1)

[2026-10-21 12:53:45] $ docker exec eco_air_active find / -name '*.java' 2>/dev/null
(exit code: 1)

[2026-10-21 12:53:45] $ docker exec eco_air_active ls -la /app
total 47800
drwxr-xr-x 1 sensor_user sensorgroup       14 Oct 21 17:53 .
drwxr-xr-x 1 root        root               6 Oct 21 17:53 ..
-rw-r--r-- 1 sensor_user sensorgroup 48944322 Oct 21 17:53 app.jar
(exit code: 0)
```

Qué demuestra: `which mvn` y `find / -name "*.java"` no devuelven nada (código de salida 1), así que no hay Maven ni código fuente en la imagen. En `/app` solo está `app.jar`.

5. Reto 3 – Límites de memoria (cgroups)

5.1 Colapso con 48 MB

```
Terminal — aleyport@fedora:~/eco-air-backend

[2026-10-21 12:53:45] $ docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0
d3e3112b9e8d7193227a485baa1b61d6c7989e28afdb4393ce639801bbe67df4
(exit code: 0)

[2026-10-21 12:53:56] $ docker inspect eco_air_crash --format='Status: {{.State.Status}} |
ExitCode: {{.State.ExitCode}} | OOMKilled: {{.State.OOMKilled}}'
Status: exited | ExitCode: 137 | OOMKilled: true
(exit code: 0)

[2026-10-21 12:53:56] $ docker inspect eco_air_crash --format='Memory limit (bytes):
{{.HostConfig.Memory}}'
Memory limit (bytes): 50331648
(exit code: 0)

[2026-10-21 12:53:56] $ docker logs --tail 25 eco_air_crash
...
... Starting EcoAirApplication v1.0.0 using Java 17.0.20.1 with PID 1
... Bootstrapping Spring Data JPA repositories in DEFAULT mode.
... Finished Spring Data repository scanning in 200 ms. Found 1 JPA repository interface.
(el log se corta aquí: el proceso fue terminado)
```

Explicación: con `-m 48m`, Docker crea un cgroup con límite de 50 331 648 bytes. Mientras Spring cargaba JPA, el proceso pasó ese límite y el **OOM killer del kernel** lo terminó con SIGKILL. Por eso el código de salida es **137 = 128 + 9** (señal 9) y Docker marca `OOMKilled: true`. La JVM no alcanzó a reaccionar: el log se corta de golpe, sin un mensaje de `OutOfMemoryError`.

5.2 Calibración del límite mínimo

Criterio: el servicio debe arrancar en menos de 15 s y seguir en estado `running` 8 s después. Primero se probó desde 96 MB y luego se afinó por debajo, de 64 a 88 MB:

Límite	Arranque	Estado tras 8 s	¿Cumple?
64 MB	No arrancó	exited · 137 · OOMKilled=true	No
72 MB	No arrancó	exited · 137 · OOMKilled=true	No
80 MB	No arrancó	exited · 137 · OOMKilled=true	No
88 MB	11.281 s	exited · 137 · OOMKilled=true	No: muere después de arrancar
96 MB	**7.298 s / 8.085 s**	**running · 0 · OOMKilled=false**	**Sí**

Umbral mínimo: 96 MB. Con 88 MB la aplicación llega a mostrar "Started", pero segundos después pasa el límite y el kernel la mata, así que no es un valor estable. Con 96 MB arranca dos veces seguidas en unos 8 s y se mantiene.

5.3 Contenedor final calibrado

```
○ ○ ○  Terminal — aleyport@fedora:~/eco-air-backend

$ docker run -d --name eco_air_active -p 8080:8080 -m 96m eco-air-backend:1.0.0
4c60da47abe2d34e27d0104f89da29a2ef76ce1d05afe25cdf60fca2ccfc6073

$ docker logs --timestamps eco_air_active | grep 'Started EcoAirApplication'
2026-10-21T19:40:15.219Z ... Started EcoAirApplication in 8.085 seconds (process running for 8.883)

$ docker stats --no-stream eco_air_active
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O       PIDS
4c60da47abe2   eco_air_active   0.15%     82.89MiB / 96MiB    86.35%    5.95kB / 126B   220MB / 311MB   31

$ docker ps -a --filter name=eco_air
CONTAINER ID   IMAGE                  STATUS                     PORTS                    NAMES
4c60da47abe2   eco-air-backend:1.0.0  Up 21 seconds              0.0.0.0:8080->8080/tcp   eco_air_active
d3e3112b9e8d   eco-air-backend:1.0.0  Exited (137) 2 hours ago                            eco_air_crash
```

Lectura de `docker stats`**:** en reposo, el contenedor usa ~83 MiB de 96 MiB (86 %), con CPU casi en cero y 31 hilos. El margen es pequeño. Es el mínimo técnico; para producción convendría dejar algo más de holgura (por ejemplo 128 MB) para picos de tráfico de los sensores.

6. Tabla de identidad

Campo	Valor
Image ID	`sha256:568e9b8ed17b3220e091a657f8347c59af87992c3f1538e49811a90b0b94fbf2`
OPERATOR_ID	`OPERATOR_ID=890275`
Label	`operator.id=890275`
Usuario	sensor_user (UID 10001)
Umbral mínimo	96 MB
docker stats (final)	82.89MiB / 96MiB · 86.35 % · CPU 0.15 %

7. Conclusiones

- **Seguridad:** la app corre como un usuario sin privilegios (UID 10001) y no puede modificar archivos del sistema. La prueba de `touch /etc/crontab_fake` lo confirmó con Permission denied.
- **Imagen limpia:** el build multietapa deja fuera Maven, el JDK y el código fuente. El historial de capas y las búsquedas con `which`/`find` no encontraron rastro de ellos.
- **Memoria:** los cgroups hacen cumplir el límite de forma estricta. Con 48 MB el kernel mata el proceso (137 / OOMKilled), y el mínimo estable para este servicio es 96 MB, gracias a que la JVM lee el límite del contenedor.
- **Mejora posible:** en el historial, `app.jar` aparece dos veces (48.9 MB en el COPY y otros 48.9 MB en el `chown`). Usar `COPY --chown=sensor_user:sensorgroup` en un solo paso evitaría esa capa duplicada y reduciría la imagen en unos 49 MB.

La evidencia completa, con hora de cada comando, está en `audit_session.log` y en `FORENSIC_REPORT.md`, dentro de `taller_docker_semana13.zip`.